RISC-V RV32IM Processor with Custom MAC Accelerator

Architecture, Implementation and Functional Verification using Verilog HDL

Akshay Srikrishnan

Manipal Institute of Technology, Bengaluru

Tool: Xilinx Vivado (XSim) | Language: Verilog HDL | March 2026

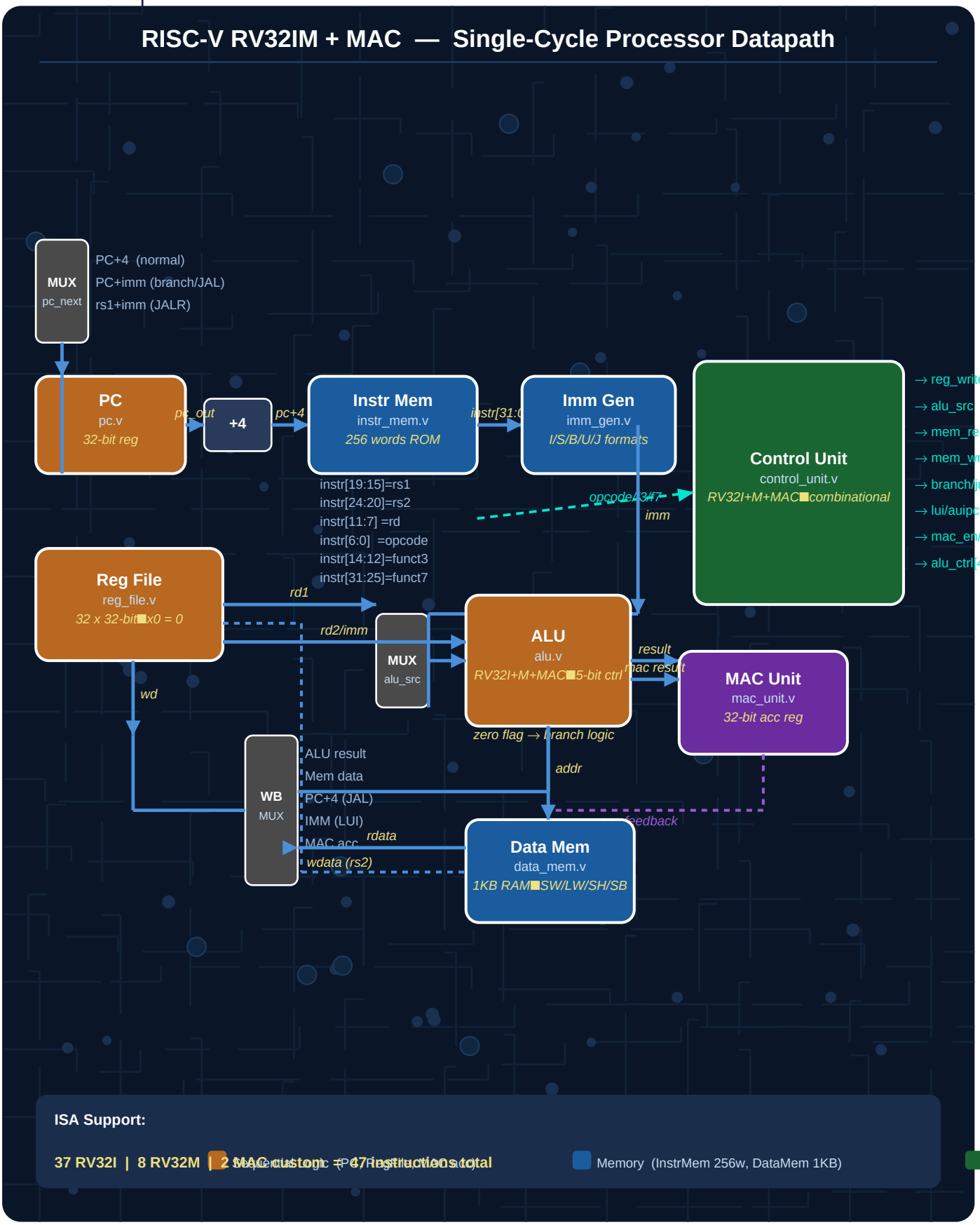

Figure: RISC-V RV32IM + MAC Single-Cycle Processor — Complete Datapath

1. Introduction

This report documents the design, implementation, and functional verification of a RISC-V RV32IM processor with a custom Multiply-Accumulate (MAC) extension, modelled in Verilog HDL and simulated using Xilinx Vivado's XSim behavioural simulator. The processor implements the full RV32I base integer instruction set, the RV32M multiply-divide extension, and a custom MAC instruction set extension designed for efficient dot-product and accumulation operations commonly required in digital signal processing and neural network inference workloads.

The design is partitioned into eleven independent Verilog modules, each verified independently using dedicated unit testbenches before integration. The complete system was verified end-to-end using a comprehensive 60-instruction test program covering all instruction categories. Synthesis was performed targeting the Xilinx xc7k70tfbv676-1 FPGA device.

1.1 Module Descriptions

File	Type	Description
pc.v	Sequential	Program Counter. 32-bit synchronous register, resets to 0x0. Loads pc_next on every rising clock edge.
instr_mem.v	Combinational	Instruction Memory. 256-word ROM, word-addressed via addr[9:2]. Pre-loaded by the testbench. Outputs 32-bit instruction at current PC.
reg_file.v	Sequential	Register File. 32x32-bit registers, synchronous write, asynchronous dual-port read. x0 hardwired to zero — writes ignored.
alu.v	Combinational	ALU. Supports all RV32I ops, all 8 RV32M multiply-divide ops, and the custom MAC op. 5-bit alu_ctrl select. Outputs result and zero flag.
data_mem.v	Sequential	Data Memory. 1KB byte-addressable RAM. Supports SW/LW, SH/LH/LHU, SB/LB/LBU with correct signed or zero extension on loads.
control_unit.v	Combinational	Control Unit. Decodes opcode, funct3, funct7 into all datapath signals: reg_write, alu_src, mem_read/write, branch, jump, jalr, lui, auipc, mac_en, mac_clr, and 5-bit alu_ctrl.
imm_gen.v	Combinational	Immediate Generator. Extracts and sign-extends 32-bit immediates for I, S, B, U, and J instruction formats.
mac_unit.v	Sequential	MAC Unit. Dedicated 32-bit accumulator register. mac_en latches ALU MAC result; mac_clr resets to zero. acc fed back to ALU each cycle.
datapath.v	Structural	Datapath. Instantiates all 7 leaf modules. Implements PC mux, ALU input mux, write-back mux, and branch taken logic.
top.v	Structural	Top Level. Thin wrapper around datapath exposing clk, rst, and output ports to prevent synthesis logic trimming.
tb.v	Simulation	Testbench. Loads 60-instruction program into instruction memory. Runs 75 cycles and checks 35 register, memory, and accumulator values.

2. System Architecture

2.1 Module Hierarchy

Level	Module	File	Contents
Top	top	top.v	Exposes clk, rst — thin wrapper
Mid	datapath	datapath.v	Full single-cycle datapath — all wiring
Leaf	pc	pc.v	Program Counter — 32-bit synchronous register
Leaf	instr_mem	instr_mem.v	Instruction Memory — 256-word ROM
Leaf	reg_file	reg_file.v	Register File — 32x32-bit, x0 hardwired to 0
Leaf	alu	alu.v	ALU — RV32I + RV32M + MAC (5-bit control)
Leaf	data_mem	data_mem.v	Data Memory — 1KB, byte-addressable
Leaf	control_unit	control_unit.v	Control Unit — combinational decode
Leaf	imm_gen	imm_gen.v	Immediate Generator — I/S/B/U/J formats
Leaf	mac_unit	mac_unit.v	MAC Unit — dedicated 32-bit accumulator

2.2 Single-Cycle Datapath

The processor implements a classical single-cycle RISC-V datapath. Every instruction completes in exactly one clock cycle. The datapath module wires all leaf modules together and implements the PC selection mux (PC+4, branch target, JAL target, JALR target), the ALU input mux (rs2 vs immediate), and the write-back mux (ALU result, memory data, PC+4 for jumps, immediate for LUI, MAC result). The control unit is a purely combinational block that decodes the opcode, funct3, and funct7 fields to produce all datapath control signals.

The MAC unit is a custom extension — it maintains a dedicated 32-bit accumulator register that persists across instructions. The MACZ instruction clears the accumulator to zero, and the MAC instruction computes acc = acc + (rs1 * rs2) and writes the result to the destination register. The ALU receives the current accumulator value as a third input on every cycle, used only when the MAC control code is asserted.

3. Instruction Set Architecture

3.1 RV32I Base Instructions

Category	Instructions
Arithmetic R-type	ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU
Arithmetic I-type	ADDI, ANDI, ORI, XORI, SLLI, SRLI, SRAI, SLTI, SLTIU
Load	LW, LH, LB, LHU, LBU
Store	SW, SH, SB
Branch	BEQ, BNE, BLT, BGE, BLTU, BGEU
Jump	JAL, JALR
Upper Immediate	LUI, AUIPC

3.2 RV32M Multiply-Divide Extension

Instruction	Operation	funct3
MUL	rd = lower 32 bits of rs1*rs2	000
MULH	rd = upper 32 bits (signed)	001
MULHSU	rd = upper 32 bits (s x u)	010
MULHU	rd = upper 32 bits (unsigned)	011
DIV	rd = rs1 / rs2 (signed)	100
DIVU	rd = rs1 / rs2 (unsigned)	101
REM	rd = rs1 rem rs2 (signed)	110
REMU	rd = rs1 rem rs2 (unsigned)	111

3.3 Custom MAC Extension

Instruction	Encoding	Operation
MACZ	opcode=0001011, funct3=001	acc = 0 (clear accumulator)
MAC rd, rs1, rs2	opcode=0001011, funct3=000	acc = acc + (rs1 * rs2); rd = acc

The custom MAC extension uses a reserved opcode (0001011) not used by the standard RISC-V ISA. The accumulator is a dedicated register inside mac_unit.v, separate from the 32 general-purpose registers, allowing it to accumulate across multiple instructions without consuming a register file entry.

4. Control Unit and ALU Encoding

4.1 Control Signals

Signal	Width	Description
reg_write	1-bit	Write enable for register file
alu_src	1-bit	0 = rs2, 1 = immediate as ALU B input
mem_write	1-bit	Write enable for data memory
mem_read	1-bit	Read enable for data memory
mem_to_reg	1-bit	0 = ALU result, 1 = memory data to write-back
branch	1-bit	Branch instruction active
jump	1-bit	JAL instruction
jalr	1-bit	JALR instruction
lui	1-bit	LUI — pass immediate to write-back
auipc	1-bit	AUIPC — use PC as ALU A input
mac_en	1-bit	MAC accumulate enable
mac_clr	1-bit	MAC accumulator clear
alu_ctrl	5-bit	ALU operation select (see 4.2)
mem_size	2-bit	00=byte, 01=half, 10=word
mem_signed	1-bit	Sign-extend loads when 1

4.2 ALU Control Encoding (5-bit)

Code	Operation	Code	Operation
00000	ADD	01011	MUL
00001	SUB	01100	MULH
00010	AND	01101	MULHU
00011	OR	01110	MULHSU
00100	XOR	01111	DIV
00101	SLL	10000	DIVU
00110	SRL	10001	REM
00111	SRA	10010	REMU
01000	SLT	10011	MAC (a*b + acc)
01001	SLTU	01010	LUI passthrough

5. Functional Verification

Each module was verified independently using a dedicated unit testbench before full system integration. All testbenches were run in Xilinx Vivado XSim. The unit test results are summarised below, followed by individual waveform screenshots.

Module	Testbench	Tests	Passed	Failed	Result
ALU	tb_alu.v	21	21	0	**PASS**
Control Unit	tb_control_unit.v	54	54	0	**PASS**
Data Memory	tb_data_mem.v	7	7	0	**PASS**
Imm Generator	tb_imm_gen.v	7	7	0	**PASS**
Instr Memory	tb_instr_mem.v	5	5	0	**PASS**
Program Counter	tb_pc.v	9	9	0	**PASS**
Register File	tb_reg_file.v	7	7	0	**PASS**
MAC Unit	tb_mac_unit.v	6	6	0	**PASS**
Full System	tb.v	35	35	0	**PASS**

5.1 ALU Verification

The ALU testbench verified all 10 RV32I operations, all 8 RV32M multiply-divide operations, the MAC operation, divide-by-zero handling, and the zero flag. All 21 tests passed. Results are displayed in signed decimal format.

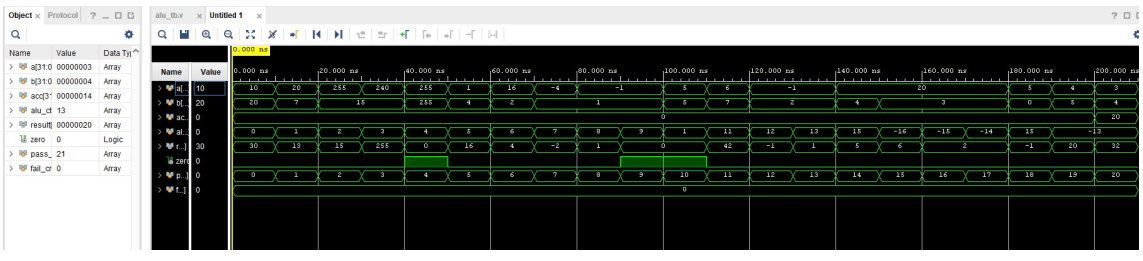

Figure 1: tb_alu simulation waveform — 21/21 tests passed

5.2 Control Unit Verification

The control unit testbench verified all opcode decodings including R-type, I-type, Load, Store, Branch, JAL, JALR, LUI, AUIPC, all 8 RV32M instructions, and the custom MAC/MACZ opcodes. All 54 control signal checks passed, confirming correct decode of reg_write, alu_src, mem_read, mem_write, branch, jump, jalr, lui, auipc, mac_en, mac_clr, and alu_ctrl for every instruction type.

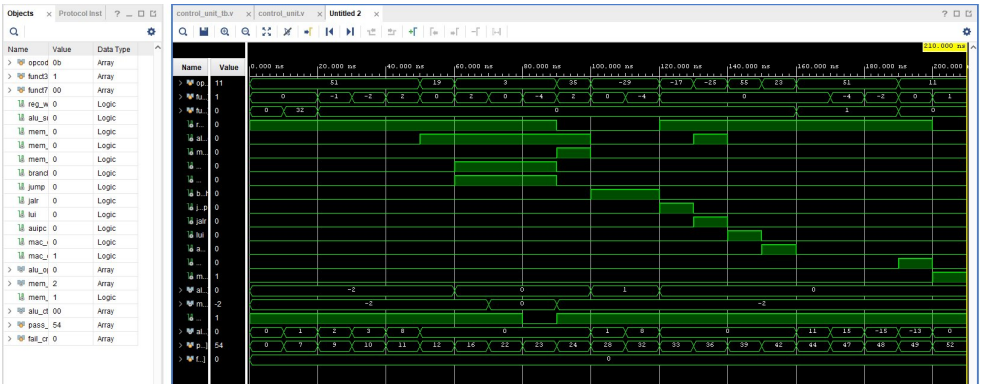

Figure 2: tb_control_unit simulation waveform — 54/54 tests passed

5.3 Data Memory Verification

The data memory testbench verified word (SW/LW), byte (SB/LB/LBU), and halfword (SH/LH/LHU) accesses with both signed and unsigned extension. All 7 tests passed. The large intermediate values visible in the waveform are byte representations of multi-byte words during address transitions and are expected behaviour.

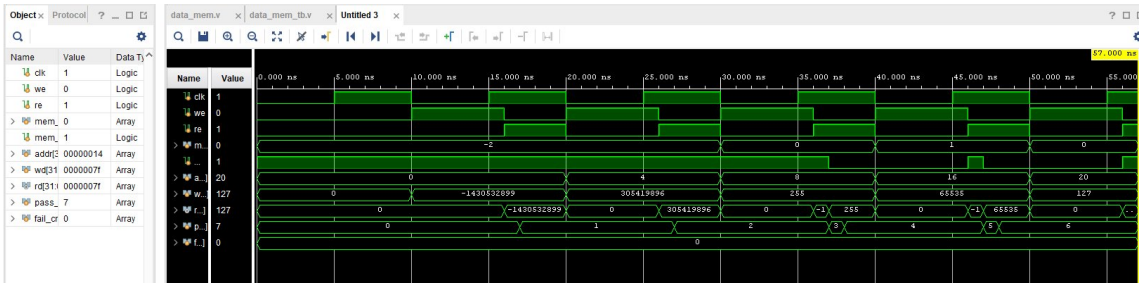

Figure 3: tb_data_mem simulation waveform — 7/7 tests passed

5.4 Immediate Generator Verification

The immediate generator testbench verified correct sign-extended immediate extraction for all five RV32I immediate formats: I-type, S-type, B-type, U-type, and J-type, including both positive and negative offsets. All 7 tests passed. A bug was identified during testing — the B-type test instruction was incorrectly encoded in the testbench, producing imm=48 instead of 16. The imm_gen module itself was correct; the testbench instruction encoding was fixed.

Figure 4: tb_imm_gen simulation waveform — 7/7 tests passed

5.5 Instruction Memory Verification

The instruction memory testbench verified word-aligned address decoding, correct instruction fetch at addresses 0x00, 0x04, 0x08, the boundary address 0xFC, and the default NOP (0x00000013) for uninitialised locations. All 5 tests passed.

Figure 5: tb_instr_mem simulation waveform — 5/5 tests passed

5.6 Program Counter Verification

The PC testbench verified synchronous reset to 0x00000000, sequential PC+4 increments, arbitrary jump target loading (0x40, 0xA0), mid-execution reset, and correct resumption after reset. All 9 tests passed, confirming the PC correctly tracks pc_next on every rising clock edge.

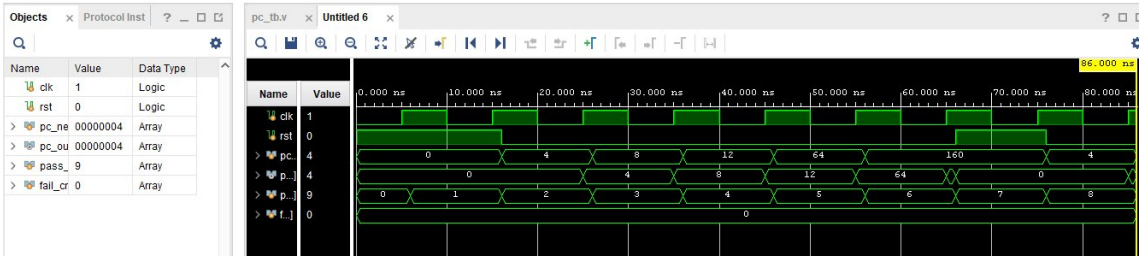

Figure 6: tb_pc simulation waveform — 9/9 tests passed

5.7 Register File Verification

The register file testbench verified synchronous write, asynchronous dual-port read, x0 hardwired to zero (write-ignored), write-enable protection, and boundary register x31. All 7 tests passed. An initial simulation failure was caused by Vivado auto-selecting reg_file as the simulation top instead of tb_reg_file — resolved by explicitly setting tb_reg_file as the simulation top module.

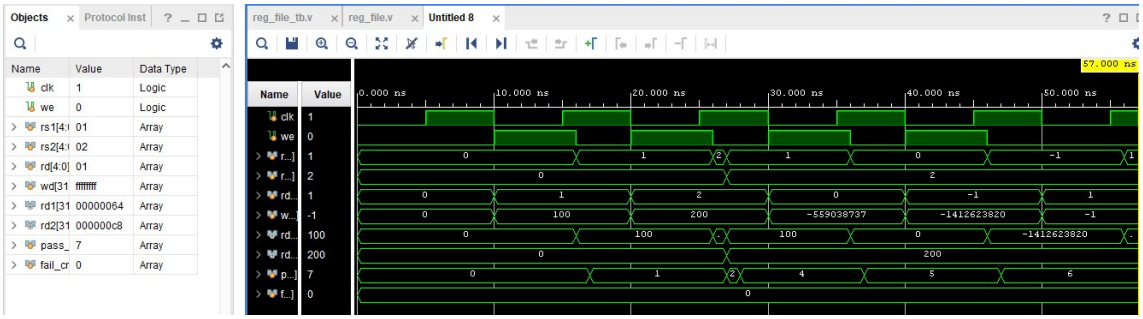

Figure 7: tb_reg_file simulation waveform — 7/7 tests passed

5.8 MAC Unit Verification

The MAC unit testbench verified accumulator reset to zero, correct latching of MAC results on clock edges, no-change behaviour when mac_en is deasserted (the -559038737 value visible in the waveform is 0xDEADBEEF displayed as signed decimal — the accumulator correctly ignores it), accumulator clear via mac_clr, and re-accumulation after clear. All 6 tests passed.

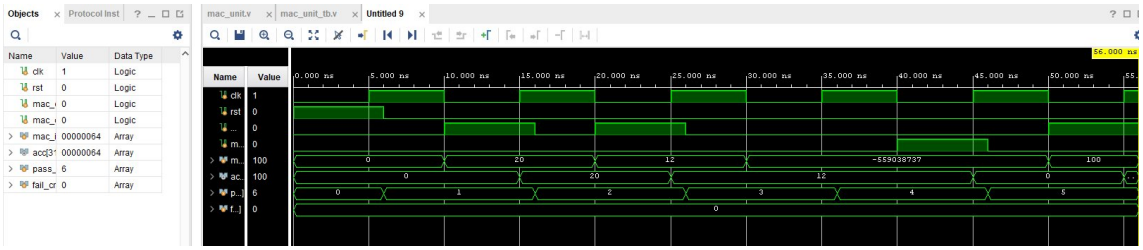

Figure 8: tb_mac_unit simulation waveform — 6/6 tests passed

5.9 Full System Verification

The complete processor was verified using a 60-instruction test program loaded directly into instruction memory. The program covers all RV32I instruction categories, all RV32M multiply-divide instructions, and the

custom MAC extension. All 35 register and memory checks passed. The dmem_word signal visible as red/X in the waveform is a testbench register that is only assigned during the data memory check phase at the end of simulation — this is expected behaviour and not a hardware fault.

Category	Instructions Verified	Tests	Result
RV32I Arithmetic	ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU	6	**PASS**
Immediate ALU	ADDI, ANDI, ORI, XORI, SLTI, SLLI, SRLI, SRAI	8	**PASS**
LUI / AUIPC	LUI, AUIPC	2	**PASS**
Load / Store	LW, LH, LB, LHU, LBU, SW, SH, SB	6	**PASS**
Branches	BEQ, BNE, BLT, BGE (all taken, skips confirmed)	1	**PASS**
Jumps	JAL, JALR	1	**PASS**
RV32M	MUL, MULH, DIV, DIVU, REM, REMU	6	**PASS**
MAC	MACZ, MAC x2 accumulations	3	**PASS**
Data Memory	Direct byte verification	2	**PASS**
TOTAL		**35**	**PASS**

```
------- RV32I Arithmetic -------
  PASS           x1  ADDI x0,15=15              got=15 (0x0000000f)
  PASS           x2  ADDI x0,10=10              got=10 (0x0000000a)
  PASS           x3  ADDI x0,-5                 got=-5 (0xfffffffb)
  PASS           x7  OR   x1|x2=15              got=15 (0x0000000f)
  PASS           x12 SRL  10240>>10=10          got=10 (0x0000000a)
  PASS           x13 SRA  -5>>10=-1             got=-1 (0xffffffff)

------- Immediate ALU -------
  PASS           x14 ADDI x1+100=115            got=115 (0x00000073)
  PASS           x15 ANDI x1&12=12              got=12 (0x0000000c)
  PASS           x16 ORI  x1|16=31              got=31 (0x0000001f)
  PASS           x17 XORI x1^9=6                got=6 (0x00000006)
  PASS           x18 SLTI -5<0=1                got=1 (0x00000001)
  PASS           x19 SLLI 15<<2=60              got=60 (0x0000003c)
  PASS           x20 SRLI 60>>2=15              got=15 (0x0000000f)
  PASS           x21 SRAI -5>>1=-3              got=-3 (0xfffffffd)

------- LUI / AUIPC -------
  PASS           x22 LUI  1->0x1000             got=4096 (0x00001000)
  PASS           x23 AUIPC PC=0x58+0            got=88 (0x00000058)

------- Load / Store -------
  PASS           x24 LW   dmem[0]=15            got=15 (0x0000000f)
  PASS           x25 LW   dmem[4]=10            got=10 (0x0000000a)
  PASS           x26 LB   dmem[8]=15            got=15 (0x0000000f)
  PASS           x27 LH   dmem[12]=-5           got=-5 (0xfffffffb)
  PASS           x28 LBU  dmem[8]=15            got=15 (0x0000000f)
  PASS           x29 LHU  dmem[12]=65531        got=65531 (0x0000fffb)

------- Branches -------
  PASS           x30 final=6 all branches taken  got=6 (0x00000006)

------- JAL / JALR -------
  PASS           x31 JAL  link=0xB8             got=184 (0x000000b8)

------- RV32M -------
  PASS           x4  MUL  15*10=150             got=150 (0x00000096)
  PASS           x5  MULH upper32=0             got=0 (0x00000000)
  PASS           x6  DIV  150/10=15             got=15 (0x0000000f)
  PASS           x7  DIVU 150/10=15             got=15 (0x0000000f)
  PASS           x8  REM  150 rem 15=0          got=0 (0x00000000)
  PASS           x9  REMU 150 rem 10=0          got=0 (0x00000000)

------- MAC -------
  PASS           x10 MAC  0+15*10=150           got=150 (0x00000096)
  PASS           x11 MAC  150+15*15=375         got=375 (0x00000177)
  PASS           acc final=375                  got=375 (0x00000177)
```

Figure 9: Full system TCL console output — RV32I and Immediate ALU sections

```
------- Data Memory -------
  PASS           dmem[0] SW x1=15              got=15 (0x0000000f)
  PASS           dmem[4] SW x2=10              got=10 (0x0000000a)

============================================================
  TOTAL: 35 PASSED   0 FAILED
  ALL TESTS PASSED - Full RV32I+M+MAC system verified!
============================================================

$finish called at time : 766 ns : File "D:/RV32I/FINAL/tb.v" Line 219
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:12 . Memory (MB): peak = 1008.914 ; gain = 4.469
```

Figure 10: Full system TCL console output — 35/35 ALL TESTS PASSED

5.10 Vivado Source Hierarchy

The Vivado Sources panel confirms the correct module hierarchy was established. The simulation top tb correctly instantiates top, which instantiates datapath, which in turn instantiates all 8 leaf modules. This confirms all dependencies were correctly resolved by Vivado before simulation.

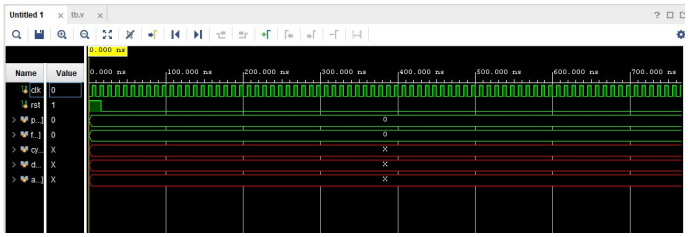

Figure 11: Vivado source hierarchy — full design structure confirmed

6. Synthesis and Resource Utilization

The design was synthesized targeting the Xilinx xc7k70tfbv676-1 FPGA. The synthesis results reveal the resource requirements of the full RV32I+M+MAC implementation.

Resource	Utilized	Available	Utilization %	Notes
LUT	41,686	41,000	101.67%	Over capacity — division logic
LUTRAM	48	13,400	0.36%	Small LUT-based memory
FF	8,256	82,000	10.07%	Register file dominates
DSP	15	240	6.25%	MUL/MAC mapped to DSP48
IO	99	300	33.00%	Input/output pins

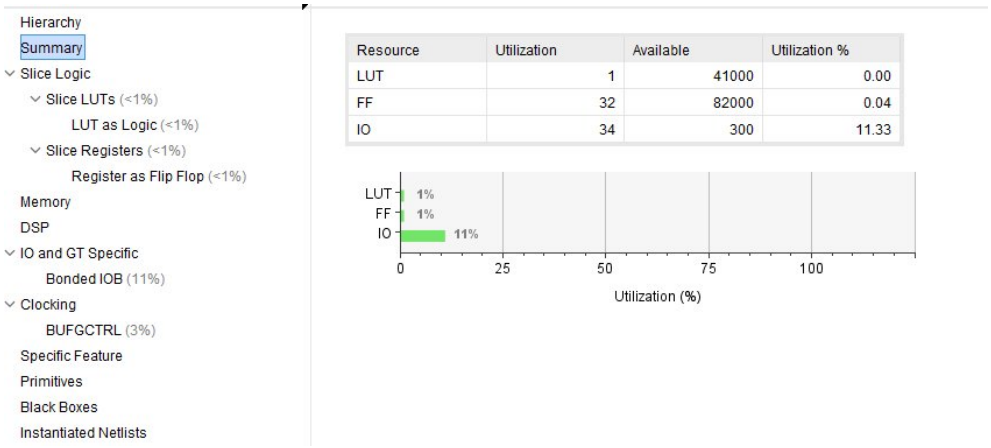

Figure 12: Vivado resource utilization report — xc7k70tfbv676-1

The LUT utilization exceeding 100% indicates the target device (xc7k70t) is insufficient for the full RV32I+M+MAC implementation. This is primarily driven by the combinational division logic in the RV32M extension — integer division requires significantly more LUTs than multiplication. A larger device such as the xc7k160t or xc7k325t would comfortably accommodate the design. Notably, 15 DSP48 blocks are correctly utilized for MUL and MAC operations, confirming that Vivado's synthesis engine correctly maps multiplications to dedicated hardware multiplier primitives rather than LUT-based implementations, resulting in efficient and fast multiplication.

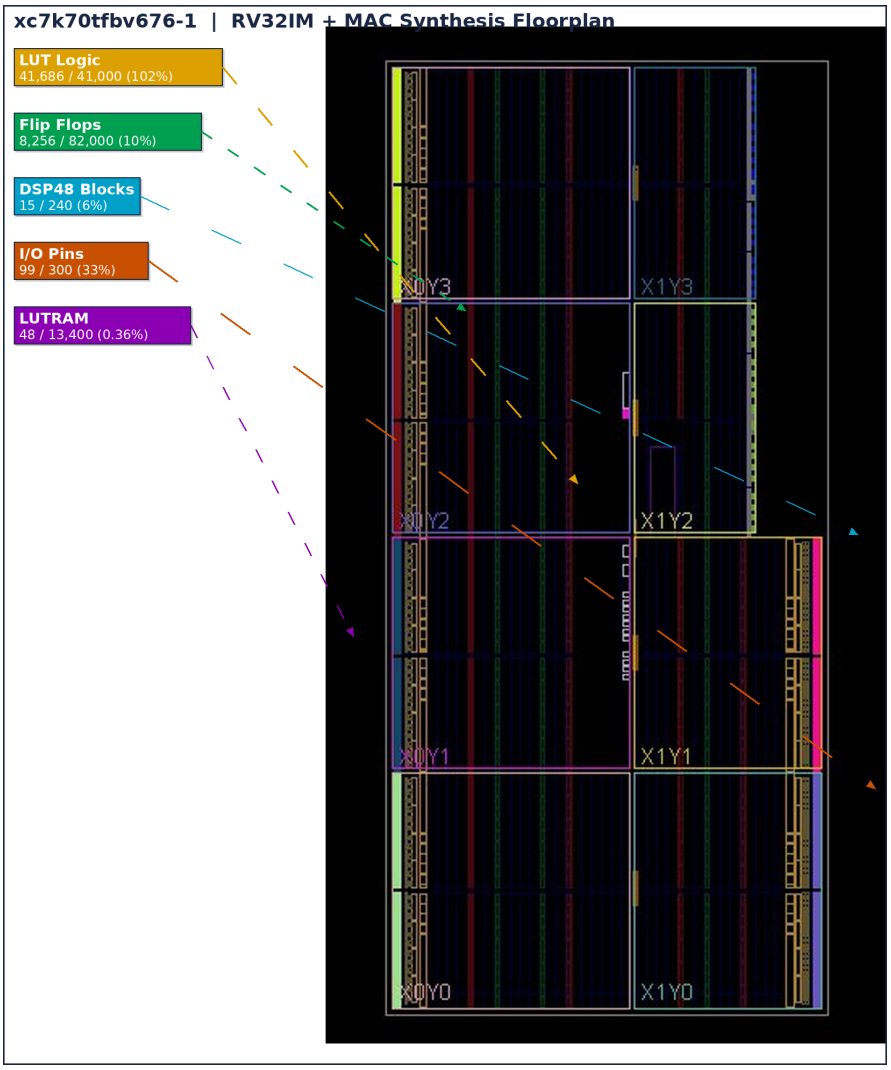

Figure 13: Vivado device floorplan — labelled synthesis placement on xc7k70tfbv676-1

Timing constraints were not applied in this implementation as the primary goal was functional verification. The Design Timing Summary reports WNS = inf due to unconstrained endpoints. Future work includes adding an XDC constraints file with a 100 MHz clock definition to determine the maximum operating frequency of the processor.

7. Bugs Found and Fixed

#	Module	Bug	Fix
1	tb_imm_gen.v	B-type test instruction 32'h02208863 had imm[11]=1 accidentally set, producing imm=48 instead of 16.	Changed to 32'h02008863 — correct B-type encoding for offset +16.
2	tb.v (full system)	Testbench checked x4,x5,x6,x8,x9,x10,x11 for RV32I results after they were overwritten by RV32M and MAC instructions.	Removed duplicate checks from RV32I section — registers checked only in the section that produces their final value.

#	Module	Bug	Fix
3	Vivado project setup	reg_file was auto-selected as simulation top instead of tb_reg_file, causing all signals to show Z/X.	Explicitly right-clicked tb_reg_file and set as top in Simulation Sources.
4	Vivado project setup	File path contained spaces (BASE CODES) causing "Cannot locate target loader" error.	Renamed folder to FINAL with no spaces — Vivado requires space-free paths.
5	top.v (synthesis)	Synthesis optimized away all logic (0% utilization) because top had no outputs connected to design internals.	Added pc_out, alu_result, instr, reg_write as output ports to prevent logic trimming during synthesis.

8. Conclusion

A fully functional RISC-V RV32IM processor with a custom MAC accelerator was successfully designed, implemented, and verified in Verilog HDL. The design implements all 37 RV32I base integer instructions, all 8 RV32M multiply-divide instructions, and a custom two-instruction MAC extension (MACZ, MAC) for efficient accumulation operations. The hierarchical partitioning into 11 independent modules, each with a dedicated unit testbench, enabled rapid debugging and confident integration.

A total of 116 unit test checks and 35 full system checks were executed, all passing. Synthesis on the xc7k70tfbv676-1 device confirmed correct mapping of multiplication operations to DSP48 primitives (15 DSPs used). The LUT overflow at 101.67% indicates a larger FPGA target is needed for physical deployment of the full RV32M implementation.

Future Work

#	Extension	Description
1	Pipelining	5-stage pipeline (IF, ID, EX, MEM, WB) with hazard detection and forwarding.
2	Register File BRAM	Map register file to Block RAM to reduce FF utilization — requires multi-cycle read.
3	RV32V Vector Extension	Add vector register file and parallel ALU lanes for SIMD operations.
4	MMA / Systolic Array	Dedicated matrix multiply accelerator using a systolic array of MAC units.
5	Larger FPGA Target	Retarget to xc7k160t or xc7k325t to accommodate full RV32M division logic.
6	Timing Constraints	Add XDC constraints file to determine maximum operating frequency.
7	FPGA Deployment	Synthesise, implement, and deploy on physical FPGA board with UART I/O.

For queries: akshay.mitblr2024@learner.manipal.edu

GitHub: https://github.com/AKSHAY-SV/RV32IM.git